MBLP131 · Hafta 15 · Çalışma Kılavuzu

İşletim Sistemleri Temelleri

Final Sınavı Tekrarı ve Alıştırma

On beş hafta, tek sınav: ilk toplu iş sistemlerinden kendi kabuk betiklerinize.

Öğretim üyesi: Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

Final sınavı için yapabilmeniz gerekenler

1. 1–7. haftaların temel fikirlerini dersin kendi terimleriyle açıklamak: işletim sisteminin bakış açıları ve hedefleri, ikili mod, sistem çağrıları, süreçler ve iş parçacıkları, zamanlama, eşzamanlama, bellek yönetimi, sanal bellek ve dosya sistemleri.
2. Standart sayısal problemleri çözmek: kullanım oranı, Amdahl yasası, bekleme ve dönüş süreleriyle Gantt şemaları, adres çevirisi, TLB ile ve sayfa hatalarıyla EAT, sayfa değiştirme çerçeve tabloları, blok hesabı.
3. 9–13. haftaların Linux komutlarını ezberden yazmak: gezinme, dosya işlemleri, izinler, yönlendirme, süreç ve iş denetimi.
4. Metin verisi (günlükler, CSV dosyaları) hakkındaki soruları yanıtlayan tek satırlık boru hatları kurmak.
5. Argüman denetimi, döngü, sınama, aritmetik ve doğru çıkış durumu içeren kısa bir Bash betiği yazmak.

İçindekiler

1 Bu Kılavuz Nasıl Kullanılır? 1

2 Kuram Tekrarı: 1–7. Haftalar 2

3 Uygulama Tekrarı: 9–14. Haftalar 5

4 Formül ve Komut Kâğıdı 10

5 Dersteki Çözümlü Problemler 10

6 Sık Yapılan Hatalar ve Sınav Stratejisi 13

7 Alıştırma Finali (100 puan) 14

8 Alıştırma Finali: Çözümler 15

9 Dersin Kapanışı ve Bundan Sonrası 20

1 Bu Kılavuz Nasıl Kullanılır?

Bu kılavuz dersin *tamamını* kapsar. Kısım 2, 1–7. kuram haftalarını kısaca tekrar eder; haftalık notlardaki terimleri, gösterimi ve uzlaşımları aynen kullanır. (8. haftanın vize çalışma kılavuzu aynı haftaları daha ayrıntılı ele alır ve hâlâ geçerlidir.) Kısım 3, 9–14. Linux haftalarını gerçek terminal çıktısıyla tekrar eder. Kısım 4 tek sayfalık bir formül ve komut kâğıdıdır. Kısım 5 ders slaytlarındaki çözümlü problemleri tam çözümleriyle yineler; Kısım 6 sık yapılan hataları ve bir sınav stratejisini listeler. Ardından eksiksiz bir **Alıştırma Finali** (Kısım 7) ve çözümleri (Kısım 8) gelir. Ayrıca dağıtılan örnek kâğıdın soruları *farklıdır*; bu kılavuzu bitirdikten sonra onu süre tutarak bir prova olarak kullanın.

1.1 Sınavla ilgili bilgiler

Köşeli parantez içindekiler henüz resmî değildir; geçerli olan, dersin Telegram kanalındaki duyurudur.

- **Tarih, saat, salon:** [dersin Telegram kanalında duyurulacak].
- **Süre:** [90] dakika. **Toplam:** altı bölümde (A–F) 100 puan.
- **Yanınızda:** öğrenci kimliği, kalem; [basit, programlanamayan hesap makinesi — serbest olup olmadığı kesinleşecek].
- **Kurallar:** kitap kapalı [kesinleşecek]; [formül kâğıdı verilir / verilmez — kesinleşecek]; telefonlar kapalı.
- **Ağırlık:** [finalin ders notuna katkısı %__ — izlenceye bakın].

Bölüm	Soru türü	Haftalar	Puan*
A	Çoktan seçmeli: kavramlar ve komutlar	1–7, 9–14	20
B	Kısa yanıt: açıkla, karşılaştır, sırala	1–7	12
C	Sayısal problemler	4–7 (+1, 3)	36
D	Komutu yazın	9–13	14
E	Boru hattını yazın	12	9
F	Kısa bir Bash betiği yazın	14	9

*Örnek kâğıdın ve aşağıdaki Alıştırma Finalinin puan dağılımı [gerçek kâğıt biraz farklı olabilir — kesinleşecek].

2 Kuram Tekrarı: 1--7. Haftalar

Her hafta için: kısa bir özet, kutu içinde formüller ve tipik sınav soruları. Buradaki bir satır size yabancı geliyorsa o haftanın notlarına dönün ve tekrar sorularını yeniden çözün.

2.1 1. hafta: Giriş ve tarihçe

Bir bilgisayar komutları izler (girdi → işlem → çıktı, program bellekte saklanır); işletim sisteminin ilk çalışma tanımı *önce başlayan ve diğer her şeyi yöneten program*dır. Tarihçe yazılımsız dönemden (ENIAC, 1945), **yerleşik monitörlü** (resident monitor) **toplu işleme** (batch processing; GM-NAA I/O, 1956), bir makine ailesi için tek işletim sistemi (IBM System/360, 1964), **çoklu programlama** (multiprogramming) ve **zaman paylaşımı** (time-sharing; CTSS 1961, Multics), UNIX (1969) ve kişisel bilgisayar üzerinden Linux'a (1991), telefonlara (iPhone 2007, Android 2008) ve buluta uzanır. Her adımı boşta kalan işlemcinin maliyeti yönlendirdi.

1. hafta formülleri

$$U = \frac{t_{\text{çalışma}}}{t_{\text{çalışma}} + t_{\text{hazırlık}}}, \qquad U(n) = 1 - p^{n} \quad (p = \text{bir işin G/Ç beklediği zaman kesri}).$$

Tipik sorular: kilometre taşlarını sıraya koyun; neden toplu iş, neden çoklu programlama, neden zaman paylaşımı; U'yu hesaplayın, ör. $t_{\text{çalışma}} = 2$ dk, $t_{\text{hazırlık}} = 3$ dk ise $U = 2/5 = 0.4$; $p = 0.6$, $n = 3$ ise $U(3) = 1 - 0.216 = 0.784$.

2.2 2. hafta: Tanımlar, hedefler, bileşenler, yapılar

İki klasik bakış: **genişletilmiş makine** (extended machine; soyutlama, yukarıdan aşağıya) olarak ve **kaynak yöneticisi** (resource manager; zamanda ve uzayda paylaştırma, aşağıdan yukarıya) olarak işletim sistemi; bellekte sürekli duran özü **çekirdektir** (kernel). Tasarım hedefleri: kolaylık, verimlilik, gelişebilirlik. Bir bilgisayar sisteminin dört bileşeni vardır: donanım, işletim sistemi, uygulama programları, kullanıcılar. Aygıtlar işlemciye **kesmelerle** (interrupt) haber verir. Bellek hiyerarşisi yazmaçlardan (≈ 0.3 ns) önbellek, RAM ($\approx$ 100 ns) ve SSD üzerinden HDD'ye ($\approx$ 5 ms) iner; yazmaçlar, önbellek ve RAM uçucudur. **İkili mod çalışması** (dual-mode operation): mod biti 0 = çekirdek modu (tüm komutlara izin var), 1 = kullanıcı modu (ayrıcalıklı komutlar yasak); bir **sistem çağrısı** (system call) çekirdek moduna geçer ve bir fonksiyon çağrısından çok daha pahalıdır. Yapılar: basit (MS-DOS), monolitik (UNIX, Linux), katmanlı (THE), mikroçekirdek (MINIX 3, QNX, seL4), modüler (yüklenebilir çekirdek modülleri), hibrit (Windows, macOS, Android). *Tipik sorular:* iki bakış açısı; koruma neden donanım ister; monolitik ve mikroçekirdeği karşılaştırın (hız ve güvenilirlik).

2.3 3. hafta: İşletim sistemi hizmetleri, sistem çağrıları, süreçler

Hizmetler kullanıcıya (kullanıcı arayüzü, program yürütme, G/Ç, dosya sistemleri, iletişim, hata algılama) ve sisteme (kaynak tahsisi, hesap tutma, koruma ve güvenlik) yardım eder. Program bir API fonksiyonunu çağırır; ince bir kütüphane **sarmalayıcısı** (wrapper) sistem çağrısı *numarasını* bir yazmaca koyar (x86-64 Linux'ta `rax`) ve tuzak (trap) komutunu çalıştırır; çekirdek bu numarayı **sistem çağrısı tablosunda** (system-call table) indeks olarak kullanır. Parametreler yazmaçlarda, bir bellek bloğunda ya da yığıtta aktarılır.

Süreç (process) yürütülmekte olan bir programdır; text, data, bss, öbek (heap) ve yığıt (stack) bölümleri vardır. Şu durumlardan geçer: *yeni*, *hazır*, *çalışıyor*, *bekliyor*, *sonlandı*; geçişler: kabul edildi, zamanlayıcı görevlendirir (dispatch), kesme, G/Ç ya da olay bekleme, G/Ç ya da olay tamamlandı, çıkış. Bekleyen bir süreç hiçbir zaman doğrudan çalışıyor durumuna geçemez. **Süreç denetim bloğu** (PCB) durumu, program sayacını, yazmaçları, zamanlama, bellek ve dosya bilgilerini saklar. **Bağlam değişimi** (context switch) bir PCB'yi kaydedip diğerini yükler ve saf ek yüktür. `fork()` bir kopya oluşturur, `exec()` yeni bir program

yükler, `wait()` bir çocuğu bekler.

3. hafta formülleri

$$\text{geçiş ek yükü} = \frac{s}{q+s}, \qquad \text{döngüde } n \text{ fork} \Rightarrow 2^n \text{ süreç } (2^n - 1 \text{ tanesini } \texttt{fork()} \text{ oluşturur}).$$

$s = 0.1$ ms ve $q = 4$ ms ile $0.1/4.1 = 0.0244$: işlemci zamanının %2,44'ü geçişte harcanır.

2.4 4. hafta: İş parçacıkları ve işlemci zamanlamasının temelleri

Bir **iş parçacığının** (thread) kendine ait iş parçacığı kimliği, program sayacı, yazmaçları ve yığıtı vardır; bir sürecin iş parçacıkları kodu, veriyi, öbeği ve açık dosyaları paylaşır. Yararları: yanıt verebilirlik, kaynak paylaşımı, ekonomi, ölçeklenebilirlik. **Eşzamanlılık** (concurrency) birden çok görevin ilerlemesi demektir; **paralellik** (parallelism) birden çok çekirdek ister. Çoklu iş parçacığı modelleri: çoktan bire, birebir (Linux, Windows), çoktan çoka. Süreçler CPU ve G/Ç patlamaları (burst) arasında gidip gelir; kısa vadeli zamanlayıcı hazır kuyruğundan seçer, dağıtıcı (dispatcher) geçişi yapar; kesintili (preemptive) zamanlayıcılar çalışan bir süreci kesebilir.

4. hafta formülleri

$$\text{hızlanma}(N) \le \frac{1}{S + \dfrac{1-S}{N}}, \qquad \lim_{N\to\infty} = \frac{1}{S};$$

$$T_{\text{dönüş}} = T_{\text{tamamlanma}} - T_{\text{geliş}}, \qquad T_{\text{bekleme}} = T_{\text{dönüş}} - T_{\text{patlama}},$$

$$T_{\text{yanıt}} = T_{\text{ilk çalışma}} - T_{\text{geliş}};$$

$$\text{işlemci kullanımı} = \frac{B}{T}, \qquad \text{üretilen iş} = \frac{n}{T}.$$

Tipik sorular: iş parçacıkları neyi paylaşır; modelleri karşılaştırın; Amdahl ($S = 0.2$, $N = 4$: $1/(0.2+0.2) = 2.5$, asla 5'ten fazla değil); ölçütler (kullanım oranını ve üretilen işi en büyük; dönüş, bekleme ve yanıt süresini en küçük yap).

2.5 5. hafta: Zamanlama algoritmaları ve eşzamanlama

FCFS (konvoy etkisi), **SJF** (tüm işler birlikte hazırsa en küçük ortalama bekleme süresi), **SRTF** (kesintili sürümü), **Öncelik** (küçük numara = yüksek öncelik; açlık, **yaşlandırma** (aging) ile giderilir), **Round Robin** (zaman dilimi q; iki sıra arası bekleme en çok $(n-1)q$; q çok büyükse $\to$ FCFS, çok küçükse $\to$ geçiş ek yükü).

Bu dersin zamanlama uzlaşımları --- sınavda belirtin

Bağlam değişimi süresi 0'dır; eşitlikler önce varış zamanına, sonra süreç adına göre bozulur; *küçük* öncelik numarası *yüksek* öncelik demektir; Round Robin'de, bir zaman diliminin dolduğu anda gelen süreç hazır kuyruğuna kesilen süreçten *önce* girer.

Yarış durumu (race condition), eşzamanlı iş parçacıkları ortak veriye eriştiğinde, en az biri yazdığında ve

sonuç zamanlamaya bağlı olduğunda ortaya çıkar (`counter++` yükle–topla–sakla demektir). **Kritik bölge problemine** (critical-section problem) bir çözüm *karşılıklı dışlamayı*, *ilerlemeyi* ve *sınırlı beklemeyi* güvence altına almalıdır. Muteks kilitleri (`acquire/release`) ve semaforlar (`wait(S)`: $S \leq 0$ olduğu sürece bekle, sonra $S{-}{-}$; `signal(S)`: $S{+}{+}$; POSIX'te `sem_wait/sem_post`) atomik donanım komutları üzerine kuruludur. Sınırlı tampon `empty`, `full` ve `mutex` kullanır; tutarsız sıralarla alınan kilitler kilitlenmeye (deadlock) yol açabilir.

5. hafta formülü

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\,\tau_n, \qquad 0 \leq \alpha \leq 1.$$

2.6 6. hafta: Bellek yönetimi

Adres bağlama derleme, yükleme ya da yürütme anında olur; modern sistemler yürütme anında bağlar; bu yüzden **mantıksal** (logical) ve **fiziksel** (physical) adresler farklıdır ve çeviriyi **MMU** yapar. Taban ve sınır: $0 \leq a < \text{sınır}$ ise $a_{\text{fiz}} = \text{taban} + a$, değilse tuzak. İlk uyan (yeten ilk delik), en iyi uyan (yeten en küçük), en kötü uyan (en büyük) ile bitişik yer ayırma **dış parçalanmadan** (external fragmentation) muzdariptir (%50 kuralı: belleğin üçte birine kadarı kaybolur); sıkıştırma yardımcı olur ama pahalıdır. **Sayfalama** (paging) mantıksal belleği sayfalara, fiziksel belleği eşit çerçevelere böler: dış parçalanma olmaz, yalnızca **iç parçalanma** (internal fragmentation) kalır (süreç başına ortalama yarım sayfa). **TLB** sayfa tablosu girdilerini önbellekte tutar. **Bölütleme** (segmentation) $\langle s, d\rangle$ adresini her bölüt için bir taban ve bir sınırla çevirir.

6. hafta formülleri

2^n baytlık sayfalarla:

$$p = \left\lfloor \frac{a}{2^n} \right\rfloor, \qquad d = a \bmod 2^n, \qquad a_{\text{fiz}} = f \cdot 2^n + d;$$

$$\text{EAT} = \alpha\,(t_{\text{TLB}} + t_m) + (1-\alpha)\,(t_{\text{TLB}} + 2t_m) = t_{\text{TLB}} + (2-\alpha)\,t_m \quad \text{(bizim uzlaşımımız).}$$

Sayfa tablosu boyutu = sayfa sayısı $\times$ girdi boyutu; ör. 32 bitlik adresler ve 4 KB'lık sayfalar için $2^{20} \times 4$ B $= 4$ MB.

2.7 7. hafta: Sanal bellek ve dosya sistemleri

İsteğe bağlı sayfalama (demand paging) bir sayfayı ilk kullanımda yükler; geçerli–geçersiz bit bellekte olmayan sayfaları işaretler; bunlardan birine dokunmak bir **sayfa hatasına** (page fault) yol açar: tuzak, sayfayı diskte bul, boş bir çerçeveye oku, tabloyu güncelle, komutu yeniden başlat. Yazınca kopyala (copy-on-write) `fork()`'u ucuzlatır. Boş çerçeve kalmadığında bir değiştirme algoritması kurban seçer: **FIFO** (en eski sayfa; Belady anomalisi olabilir), **OPT** (gelecekte en geç kullanılacak olan; en iyisidir ama geleceği bilmeyi gerektirir), **LRU** (en uzun süredir kullanılmayan; saat/ikinci şans algoritmasıyla yaklaşık olarak gerçekleştirilir). **Boşa çalışma** (thrashing): süreçlerin çerçeve sayısı çalışma kümelerinden azdır ve zamanlarını sayfalamayla geçirirler. Dosyalar öznitelikleri olan adlandırılmış verilerdir (Linux'ta düğüm, inode); erişim sıralı, doğrudan ya da indekslidir. Dizinler: tek düzeyli, iki düzeyli, ağaç, döngüsüz çizge (sabit ve sembolik bağlantılar), genel çizge.

7. hafta formülleri

$$\text{EAT} = (1-p)\,t_m + p\,t_{\text{fault}}; \qquad \text{bayt} = r \times \text{kayıt boyutu}, \quad \text{blok} = \left\lfloor \frac{\text{bayt}}{B} \right\rfloor, \quad \text{konum} = \text{bayt mod } B.$$

3 Uygulama Tekrarı: 9--14. Haftalar

Çıktılar hakkında. Bu kısımdaki her terminal oturumu gerçek çıktıdır; sıradan bir kullanıcı olarak, yeni oluşturulmuş bir `~/lab` klasöründe kaydedildi (önce `bash mcmp117_lab_setup.sh` çalıştırın; betik `code/` klasöründedir). Komutlar `code/demo_recap.sh`, `code/demo_worked.sh` ve `code/answers_*.sh` dosyalarında, çıktılar `code/out_*.txt` dosyalarındadır. Sizin makinenizde tarihler, boyutlar, PID'ler ve kullanıcı adları farklı olacaktır; `ls` ve `sort` çıktısının sırası yerel ayara (locale) bağlı olabilir; biz `LC_ALL=C.UTF-8` kullandık. Türkçe yerel ayarlı bir sistemde (`LANG=tr_TR.UTF-8`) birçok ileti Türkçe görünür ve sıralama farklı olabilir; komutlar ve seçenekleri ise aynıdır.

3.1 9. hafta: Kabuk temelleri ve gezinme

Terminal penceredir; *kabuk* (shell; bash) komutları okuyan bir sistem programıdır; işi sistem çağrıları aracılığıyla çekirdek yapar (2. hafta). Bir komutun biçimi `komut -seçenekler argümanlar` şeklindedir. `pwd` çalışma dizinini (working directory) yazdırır, `ls` listeler (`-l` uzun, `-a` gizli, `-h` okunabilir boyutlar, `-R` özyinelemeli), `cd` dizin değiştirir. **Mutlak yollar** (absolute path) `/` ile başlar; **göreli yollar** (relative path) çalışma dizininden başlar; `~` ev dizini, `.` burası, `..` üst dizin, `cd -` geri döner. Önemli dizinler (FHS): `/home`, `/etc`, `/bin`, `/usr`, `/var`, `/tmp`, `/dev`, `/proc`. Yardım: `man` (`q` çıkar, `/sözcük` arar), `--help`. Sekme tamamlama, `history`, `!!`, Ctrl+C, Ctrl+L.

```
student@okan-lab:~/lab$ pwd
/home/student/lab
student@okan-lab:~/lab$ ls
README.txt  backup  data  docs  logs  photos  projects  scripts  src
student@okan-lab:~/lab$ cd projects/alpha/src
student@okan-lab:~/lab/projects/alpha/src$ pwd
/home/student/lab/projects/alpha/src
student@okan-lab:~/lab/projects/alpha/src$ cd ../../beta/docs
student@okan-lab:~/lab/projects/beta/docs$ pwd
/home/student/lab/projects/beta/docs
student@okan-lab:~/lab/projects/beta/docs$ cd ~/lab
```

3.2 10. hafta: Dosya ve dizin işlemleri

`mkdir -p`, `touch`, `cp` (`-r -i -v`), `mv` (yeniden adlandır ya da taşı), `rm` (`-r -i`; **geri dönüşüm kutusu yoktur**: Enter'a basmadan önce yolu denetleyin), `rmdir`. Joker karakterler (wildcards) `*` (herhangi bir dizgi), `?` (tam olarak tek karakter), `[abc]`, `[0-9]` ve süslü ayraç genişletmesi (brace expansion) `{a,b}`, `{1..5}`, komut çalışmadan önce *kabuk tarafından* genişletilir. Boşluklu adları tırnaklayın. Görüntüleme: `cat`, `less`, `head`, `tail -n/-f`, `wc`, `file`, `stat`; boyutlar `du -sh`, `df -h`; bağlantılar `ln` (sabit) ve `ln -s` (sembolik), düğüm numaraları `ls -i` ile (7. hafta).

```
student@okan-lab:~/lab$ ls photos/img00?.jpg
photos/img001.jpg  photos/img004.jpg  photos/img007.jpg
photos/img002.jpg  photos/img005.jpg  photos/img008.jpg
photos/img003.jpg  photos/img006.jpg  photos/img009.jpg
student@okan-lab:~/lab$ echo photos/*.{png,JPG}
photos/img013.png photos/IMG_014.JPG
student@okan-lab:~/lab$ mkdir -p backup/{2025,2026}
student@okan-lab:~/lab$ cp docs/notes_0*.txt backup/2026/
student@okan-lab:~/lab$ ls backup/2026
notes_01.txt  notes_02.txt  notes_03.txt
student@okan-lab:~/lab$ wc -l data/students.csv docs/syllabus.txt
  41 data/students.csv
  15 docs/syllabus.txt
  56 total
student@okan-lab:~/lab$ ln -s ../logs/access.log docs/latest.log
student@okan-lab:~/lab$ ls -l docs/latest.log
lrwxrwxrwx 1 student student 18 Oct 12 08:50 docs/latest.log -> ../logs/access.log
```

3.3 11. hafta: İzinler, sahiplik ve kullanıcılar

`-rwxr-x---` mod dizgisinde bir tür karakteri ve sahip (u), grup (g) ve diğerleri (o) için üç üçlü vardır. Dosyalar için r/w/x okuma/yazma/çalıştırma anlamına gelir; dizinler için r listeler, w girdi oluşturur ve siler, x içine girer (`cd`). Sekizli gösterim: $r = 4$, $w = 2$, $x = 1$; dolayısıyla `rwxr-x---` $= 750$ ($7 = 4+2+1$, $5 = 4+1$, 0). `chmod` sembolik (`u+x`, `go-w`, `a =r`) ya da sekizli kullanılır. **umask** istenen moddan (dosyalar için 666, dizinler için 777) bit *siler*: yeni mod $=$ istenen mod AND NOT umask (bit düzeyinde; çıkarma değil); umask 002 $664/775$, umask 027 $640/750$ verir. `chown`/`chgrp` root yetkisi ister; `sudo` tek bir komutu root olarak çalıştırır — yazmadan önce düşünün. Bir izin sorununu asla `chmod 777` ile "çözmeyin": bu, her kullanıcının dosyayı değiştirmesine izin verir.

```
student@okan-lab:~/lab$ id
uid=1015(student) gid=1017(student) groups=1017(student)
student@okan-lab:~/lab$ touch scripts/run.sh
student@okan-lab:~/lab$ ls -l scripts/run.sh
-rw-rw-r-- 1 student student 0 Oct 12 08:50 scripts/run.sh
student@okan-lab:~/lab$ chmod 750 scripts/run.sh
student@okan-lab:~/lab$ ls -l scripts/run.sh
-rwxr-x--- 1 student student 0 Oct 12 08:50 scripts/run.sh
student@okan-lab:~/lab$ chmod g-x,o+r scripts/run.sh
student@okan-lab:~/lab$ ls -l scripts/run.sh
-rwxr--r-- 1 student student 0 Oct 12 08:50 scripts/run.sh
student@okan-lab:~/lab$ umask
0002
```

3.4 12. hafta: Metin işleme, yönlendirme ve borular

Her süreç üç açık dosya tanımlayıcısıyla başlar: 0 standart girdi (stdin), 1 standart çıktı (stdout), 2 standart hata (stderr) (2–3. haftalar: `write(1, …)`). **Yönlendirme** (redirection): `>` üzerine yaz, `>>` sona ekle, `<` girdi, `2>` hatalar, `2>&1` hataları stdout'un gittiği yere, `&>` ikisi birden, `/dev/null` atar. Bir **boru** (pipe) `a | b`, a'nın stdout'unu b'nin stdin'ine bağlar; iki süreci de kabuk oluşturur (3. hafta). Süzgeçler: `grep` (`-i -n -v -c -r -w`; düzenli ifadeler `^ $ . [ ] *`), `sort` (`-n -r -t -k`), `uniq` (`-c`; sıralı girdi ister), `cut` (`-d -f`), `wc`,

`tr`, `head`, `tail`, `tee`, `find` (`-name -type -size -mtime`), `xargs`.

```
student@okan-lab:~/lab$ grep -c ERROR logs/system.log
13
student@okan-lab:~/lab$ grep -n ERROR logs/system.log | head -3
13:2026-10-12 09:12:00 ERROR failed to mount /dev/sdb1
17:2026-10-12 09:56:00 ERROR failed to mount /dev/sdb1
18:2026-10-12 09:07:00 ERROR failed to mount /dev/sdb1
student@okan-lab:~/lab$ cut -d' ' -f9 logs/access.log | sort | uniq -c | sort -rn
    148 200
     57 404
     34 304
     24 500
     20 301
     17 403
student@okan-lab:~/lab$ ls nosuchfile 2> errors.txt
student@okan-lab:~/lab$ cat errors.txt
ls: cannot access 'nosuchfile': No such file or directory
```

`logs/access.log` dosyasında boşlukla ayrılmış alanlar şunlardır: 1 IP adresi, 4–5 tarih ve saat dilimi, 6–8 istek (yöntem, sayfa, protokol), 9 durum kodu, 10 bayt sayısı. `data/students.csv` dosyasında (başlık `id,name,department,midterm,final`) virgülle ayrılmış alanlar 1–5'tir.

3.5 13. hafta: Süreçler ve iş denetimi

`ps`, `ps -ef`, `ps aux` (PID, PPID, STAT, %CPU, %MEM sütunları), `pstree`, `top`/`htop`, `/proc/<pid>/status`. STAT 3. haftanın durumlarını gösterir: R çalışıyor ya da hazır, S uyuyor (bekliyor), T durdurulmuş, Z zombi. **İşler** (jobs): `cmd &` arka planda (background) çalıştırır, `jobs` listeler, `fg`/`bg` ön plana (foreground) ya da arka plana taşır, Ctrl+Z durdurur (SIGTSTP), Ctrl+C keser (SIGINT). **Sinyaller** (signals): `kill PID` SIGTERM gönderir (15, "lütfen dur"), `kill -9` SIGKILL gönderir (yakalanamaz, yok sayılamaz — süreç temizlik yapma fırsatı bulamaz: yalnızca SIGTERM işe yaramazsa kullanın), SIGSTOP (19) ve SIGCONT (18); `pkill`, `killall`. `$$` kabuğun, `$!` son arka plan işinin PID'idir. `nice`/`renice`: -20 (en yüksek öncelik) ile 19 arası değerler; bunları yalnızca root düşürebilir (5. hafta).

```
student@okan-lab:~/lab$ sleep 600 &
student@okan-lab:~/lab$ echo $!
23441
student@okan-lab:~/lab$ jobs
[1]+  Running                 sleep 600 &
student@okan-lab:~/lab$ ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
23361 23360 Ss+  bash /home/student/w15/demo_recap.sh
23441 23361 S+   sleep 600
23445 23361 R+   ps -o pid,ppid,stat,cmd
student@okan-lab:~/lab$ kill %1
student@okan-lab:~/lab$ kill -l TERM KILL INT STOP CONT
15
9
2
19
18
```

İlk `ps` satırı komutlarımızı çalıştıran kabuktur (burada bir betik; sizin terminalinizde `bash` göreceksiniz). Etkileşimli bir terminalde kabuk bir sonraki istemde ayrıca `[1]+ Terminated sleep 600` bildirir.

3.6 14. hafta: Kabuk betikleri

Bir **betik** (script) `#!/bin/bash` shebang satırıyla başlar; `chmod +x s.sh; ./s.sh` ya da `bash s.sh` ile çalıştırılır; `bash -x` ile hata ayıklanır. **Değişkenler** (variables): `x=5` (= çevresinde boşluk yok); `"$x"` genişler, `'$x'` genişlemez; komut yerine koyma `$(cmd)`; tamsayı aritmetiği `$((x + 1))`. Konumsal parametreler `$0 $1 $# $@`; `$?` son komutun çıkış durumudur, `exit n` betiğin kendi çıkış durumunu belirler. Sınamalar `[ … ]` (köşeli ayraçların içinde boşluk!): sayılar `-eq -ne -lt -le -gt -ge`, dizgiler `= != -z`, dosyalar `-f -d -e`. Denetim: **koşul** `if/elif/else/fi`, **döngüler** `for … in …; do …; done` ve `while`, fonksiyonlar. 14. haftada `backup.sh` ve `report.sh` betiklerini yazdık.

```
student@okan-lab:~/lab$ name=student; echo "Hello $name"
Hello student
student@okan-lab:~/lab$ echo 'Hello $name'
Hello $name
student@okan-lab:~/lab$ echo "2^10 = $((2 ** 10))"
2^10 = 1024
student@okan-lab:~/lab$ n=$(wc -l < data/students.csv); echo "students.csv has $n lines"
students.csv has 41 lines
student@okan-lab:~/lab$ [ -d logs ] && echo "logs is a directory"
logs is a directory
student@okan-lab:~/lab$ for c in 1 2 3; do echo "item $c"; done
item 1
item 2
item 3
```

4 Formül ve Komut Kâğıdı

Hafta	Büyüklük	Formül
1	Toplu iş kullanımı	$U = t_{\text{çalışma}}/(t_{\text{çalışma}} + t_{\text{hazırlık}})$
1	Çoklu programlama	$U(n) = 1 - p^n$
3	Geçiş ek yükü; fork	$s/(q+s)$; döngüde n fork $\Rightarrow 2^n$ süreç
4	Amdahl yasası	hızlanma $\leq 1/\big(S + (1-S)/N\big)$, sınır $1/S$
4	Süreler	$T_{\text{dönüş}} = T_{\text{tamamlanma}} - T_{\text{geliş}}$; $T_{\text{bekleme}} = T_{\text{dönüş}} - T_{\text{patlama}}$; $T_{\text{yanıt}} = T_{\text{ilk çalışma}} - T_{\text{geliş}}$
4	Kullanım, üretilen iş	$B/T,\ n/T$
5	Patlama tahmini; RR	$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$; iki sıra arası bekleme $\leq (n-1)q$
6	Taban/sınır	$0 \leq a <$ sınır $\Rightarrow a_{\text{fiz}} =$ taban $+ a$, değilse tuzak
6	Sayfalama	$p = \lfloor a/2^n \rfloor$, $d = a \bmod 2^n$, $a_{\text{fiz}} = f \cdot 2^n + d$
6	TLB	EAT $= t_{\text{TLB}} + (2-\alpha)\,t_m$ (sadeleştirilmiş, $t_{\text{TLB}} = 0$: $\alpha t_m + (1-\alpha)2t_m$)
7	Sayfa hataları	EAT $= (1-p)\,t_m + p\,t_{\text{fault}}$
7	Doğrudan erişim	bayt $= r\times$ boyut; blok $= \lfloor$bayt$/B\rfloor$; konum $=$ bayt mod B

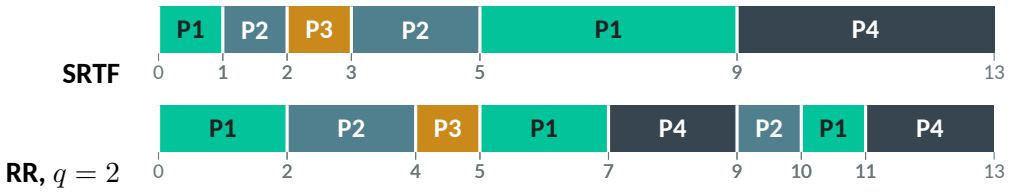
Görev	Komut	Görev	Komut
Neredeyim?	`pwd`	Satır bul	`grep -n -i -v -c -w`
Uzun, gizliler dahil	`ls -la  (-h)`	k. alan	`cut -d, -f3`
Dizin değiştir	`cd ~  cd -  cd ..`	Sırala	`sort -n -r -t, -k5`
Klasör oluştur	`mkdir -p a/b`	Tekrarları say	`sort | uniq -c`
Kopyala / taşı	`cp -r a b  mv a b`	Başlığı atla	`tail -n +2 f`
Sil	`rm -i f  rm -r dir`	Dosya bul	`find . -name "*.c"`
Bağlantılar	`ln f h  ln -s f s`	Yönlendir	`> >> < 2> 2>&1 &>`
Dosyaya bak	`cat less head tail`	Süreçler	`ps aux  ps -ef  top`
Say, boyutlar	`wc -l  du -sh  df -h`	İşler	`cmd &  jobs  fg %1  bg`
İzinler	`chmod 640 f  chmod u+x f`	Sinyaller	`kill PID  kill -9 PID`
Kullanıcılar	`whoami  id  groups`	Öncelik	`nice -n 10 cmd`
Sahip (root)	`sudo chown u:g f`	Betik	`#!/bin/bash  $1 $# $?`
Yardım	`man ls  ls --help`	Aritmetik	`$((x + 1))  $(cmd)`

5 Dersteki Çözümlü Problemler

Aşağıdaki tüm sayılar `code/theory_check.py` ile hesaplandı (çıktısı `code/out_theory_check.txt` dosyasında).

5.1 Problem 1: SRTF ve Round Robin

P1 (varış 0, patlama 5), P2 (1, 3), P3 (2, 1), P4 (3, 4); süreler ms.



SRTF: $t = 1$'de P2 (3) P1'i (4 kaldı) keser; $t = 2$'de P3 (1) P2'yi keser; P2 5'te biter; sonra P1 ile P4'ün ikisi de 4 ms ister ve eşitlik P1'in lehine bozulur (daha erken varış). Bekleme süreleri $4, 1, 0, 6$: ortalama $11/4 = 2.75$ ms; dönüş süreleri $9, 4, 1, 10$: ortalama 6 ms. RR: $t = 2$'de kuyruk P2, P3, P1'dir (P3 tam 2'de geldi ve kesilen P1'den önce girer). Bekleme $6, 6, 2, 6$: ortalama 5 ms; dönüş $11, 9, 3, 10$: ortalama 8.25 ms; yanıt $0, 1, 2, 4$: ortalama 1.75 ms (SRTF: 1.5 ms).

5.2 Problem 2: adres çevirisi ve TLB

1 KB $= 2^{10}$ baytlık sayfalar; sayfa tablosu $0 \to 5$, $1 \to 2$, $2 \to 7$, $3 \to 1$. Mantıksal adres 2500: $p = \lfloor 2500/1024 \rfloor = 2$, $d = 2500 - 2048 = 452$, çerçeve 7, $a_{\text{fiz}} = 7 \times 1024 + 452 = 7620$. $t_{\text{TLB}} = 10$ ns, $t_m = 100$ ns, $\alpha = 0.9$ ile: EAT $= 10 + 1.1 \times 100 = 120$ ns ($= 0.9 \times 110 + 0.1 \times 210$). EAT $= 115$ ns için: $10 + (2 - \alpha)\,100 = 115 \Rightarrow \alpha = 0.95$.

5.3 Problem 3: sayfa değiştirme

3 çerçeveyle $3, 5, 1, 0, 3, 1, 0, 3, 1, 4, 1, 0$ başvuru dizisi: FIFO 8, LRU 7, OPT 5 hata. LRU tablosu (kalın = yeni yüklenen sayfa):

başv.	**3**	**5**	**1**	**0**	**3**	**1**	**0**	**3**	**1**	**4**	**1**	**0**
çr. 0	**3**	3	3	**0**	0	0	0	0	0	**4**	4	4
çr. 1		**5**	5	5	**3**	3	3	3	3	3	3	**0**
çr. 2			**1**	1	1	1	1	1	1	1	1	1
hata	**F**	**F**	**F**	**F**	**F**					**F**		**F**

OPT dördüncü başvuruda (0) bir daha hiç kullanılmayan 5'i, 4 başvurusunda da bir daha gerekmeyen 3'ü çıkarır: 5 hata, bunların üçü zorunlu hatadır.

5.4 Problem 4: kısa hesaplar

- $t_m = 100$ ns, $t_{\text{fault}} = 10$ ms $= 10^7$ ns, $p = 0.0002$: EAT $= 0.9998 \times 100 + 0.0002 \times 10^7 = 99.98 + 2000 = 2099.98$ ns, yani RAM'den 21 kat yavaş. Yavaşlamanın %10'un altında kalması için: $p < 10/9\,999\,900 \approx 1.0 \times 10^{-6}$.
- Kayıt 75, 128 B'lık kayıtlar, 4096 B'lık bloklar: bayt 9600, blok $\lfloor 9600/4096 \rfloor = 2$, konum $9600 - 8192 = 1408$; son bayt 9727 hâlâ blok 2'dedir.
- $U(3) = 1 - 0.6^3 = 0.784$; toplu iş $2/(2+3) = 0.4$; ek yük $0.1/4.1 = 0.0244$ (%2,44); Amdahl $1/(0.2 + 0.8/4) = 2.5$.

5.5 Problem 5--7: komut, boru hattı, betik

İzinler. `rwx r-x ---` $= 750$, dolayısıyla `chmod 750 report.sh`. `umask 027`'den sonra $666/777$'den 0, 2, 7 bitleri silinir: yeni dosyalar 640, yeni dizinler 750.

```
student@okan-lab:~/lab$ touch report.sh
student@okan-lab:~/lab$ ls -l report.sh
-rw-rw-r-- 1 student student 0 Oct 12 08:50 report.sh
student@okan-lab:~/lab$ chmod 750 report.sh
student@okan-lab:~/lab$ ls -l report.sh
-rwxr-x--- 1 student student 0 Oct 12 08:50 report.sh
student@okan-lab:~/lab$ umask 027
student@okan-lab:~/lab$ touch new.txt; mkdir newdir
student@okan-lab:~/lab$ ls -ld new.txt newdir
-rw-r----- 1 student student    0 Oct 12 08:50 new.txt
drwxr-x--- 2 student student 4096 Oct 12 08:50 newdir
```

`access.log` üzerinde boru hatları. İlk-N kalıbı: `cut | sort | uniq -c | sort -rn | head`. 404 sayısı için önce durum alanını kesin: `404` bir bayt sayısının içinde de geçebilir.

```
student@okan-lab:~/lab$ cut -d' ' -f1 logs/access.log | sort | uniq -c | sort -rn | head -3
     58 10.0.2.17
     56 10.0.4.31
     52 10.0.6.45
student@okan-lab:~/lab$ cut -d' ' -f9 logs/access.log | grep -c '^404$'
57
student@okan-lab:~/lab$ grep ' 404 ' logs/access.log | cut -d' ' -f7 | sort | uniq -c | sort -
    rn | head -1
      9 /images/logo.png
```

`dept_stats.sh` betiği. `$((sum / count))` içindeki tamsayı bölmesine dikkat: $1072/17 = 63.06$, 63 olarak yazdırılır.

```bash
#!/bin/bash
# dept_stats.sh -- number of students and average final grade of ONE department
# usage: ./dept_stats.sh DEPARTMENT        (run inside ~/lab)
if [ $# -ne 1 ]; then
    echo "usage: $0 DEPARTMENT" >&2
    exit 1
fi
dept=$1
count=0
sum=0
for f in $(grep ",$dept," data/students.csv | cut -d, -f5); do
    count=$((count + 1))
    sum=$((sum + f))
done
if [ $count -eq 0 ]; then
    echo "no students in $dept"
    exit 2
fi
echo "$dept: $count students, average final $((sum / count))"
```

```
student@okan-lab:~/lab$ chmod u+x scripts/dept_stats.sh
student@okan-lab:~/lab$ ./scripts/dept_stats.sh ECT
ECT: 17 students, average final 63
student@okan-lab:~/lab$ ./scripts/dept_stats.sh CS
CS: 10 students, average final 65
student@okan-lab:~/lab$ ./scripts/dept_stats.sh
usage: ./scripts/dept_stats.sh DEPARTMENT
student@okan-lab:~/lab$ echo $?
1
```

6 Sık Yapılan Hatalar ve Sınav Stratejisi

Bekleme ve dönüş Bekleme = dönüş − patlama; dönüş süresi 0'dan değil, *varıştan* başlar.

Round Robin kuyruğu Zaman diliminin dolduğu anda gelen süreç, kesilen süreçten önce kuyruğa girer.

Birimler 8 ms $= 8\,000\,000$ ns. Önce çevirin, sonra yerine koyun.

TLB ıskası Bir ıska iki bellek erişimine (sayfa tablosu ve veri) ve TLB aramasına mal olur.

LRU ve OPT LRU geriye bakar, OPT ileriye bakar.

Blok hesabı Kayıtlar ve bloklar 0'dan numaralanır; son baytın bir sonraki bloğa geçip geçmediğine bakın.

`sort` olmadan `uniq` `uniq -c` yalnızca *ardışık* eşit satırları birleştirir.

Boşluklar `x =5`, `x = 5` değil ("command not found"); `[ $a -gt 3 ]`, `[$a -gt 3]` değil.

Tırnaklar `"$x"` genişler, `'$x'` genişlemez; boşluklu adları tırnaklayın: `"final report.txt"`.

> ve >> > üzerine yazar, >> sona ekler, 2> yalnızca hataları yönlendirir.

Fazla güçlü araçlar `chmod 777`, `rm -r`, `kill -9`: fazla izin, geri alma yok, temizlik yok.

Sınav stratejisi

- **Önce:** 1–7. haftaların tekrar sorularını ve 9–14. haftaların laboratuvar föylerini yeniden çözün; aşağıdaki Alıştırma Finalini süre tutarak kâğıt üzerinde, ardından örnek kâğıdı çözün; komut kâğıdındaki her komutu `~/lab` içinde yazın.
- **Sırasında:** önce bütün kâğıdı okuyun (5 dk); en iyi bildiğinizle başlayın; puan başına yaklaşık bir dakika düşün, bu yüzden A ve D bölümlerini hızlı yapıp zamanı C bölümüne harcayın; formül $\rightarrow$ sayılar $\rightarrow$ birimiyle sonuç biçiminde yazın; kullandığınız uzlaşımları belirtin; hiçbir soruyu boş bırakmayın.

7 Alıştırma Finali (100 puan)

Çözümleri (Kısım 8) okumadan önce bu kâğıdı kendi başınıza, [90] dakikada çözün. Komutlar `mcmp117_lab_setup.sh` ile oluşturulan `~/lab` klasörüne göredir; soruda aksi belirtilmedikçe çalışma dizininizin `~/lab` olduğunu varsayın.

Bölüm A --- Çoktan seçmeli ($10 \times 2 = 20$ puan)

A1. Hangisi 2. haftadaki üç işletim sistemi tasarım hedefinden biri *değildir*? (a) kolaylık (b) verimlilik (c) gelişebilirlik (d) en yüksek donanım maliyeti

A2. Çekirdek modunda mod biti (a) 0 (b) 1 (c) 2 (d) tanımsız

A3. G/Ç'si tamamlanan bekleyen bir süreç hangi duruma geçer? (a) çalışıyor (b) hazır (c) yeni (d) sonlandı

A4. Bir sürecin iş parçacıkları hangisini *paylaşır*? (a) yığıt (b) yazmaçlar (c) öbek (d) program sayacı

A5. Hangi algoritmada Belady anomalisi görülebilir? (a) OPT (b) LRU (c) FIFO (d) hiçbiri

A6. TLB neyi önbellekte tutar? (a) disk bloklarını (b) sayfa tablosu girdilerini (c) PCB'leri (d) dosya adlarını

A7. Bir dizinin modu `drwxr-x---`. Grubunun bir üyesi (a) içinde dosya oluşturabilir (b) listeleyip içine girebilir (c) yalnızca içine girebilir (d) hiçbir şey yapamaz

A8. Hangi yönlendirme standart çıktıyı bir dosyanın *sonuna ekler*? (a) > (b) >> (c) 2> (d) <

A9. Hangi sinyal bir süreç tarafından ne yakalanabilir ne de yok sayılabilir? (a) SIGTERM (b) SIGINT (c) SIGKILL (d) SIGTSTP

A10. Bir Bash betiğinde `$#` nedir? (a) betiğin adı (b) PID (c) argüman sayısı (d) son çıkış durumu

Bölüm B --- Kısa yanıt ($3 \times 4 = 12$ puan)

B1. Program ile süreç arasındaki fark nedir? Beş süreç durumunu ve bir süreci çalışıyor durumundan bekliyor durumuna geçiren olayı yazın. **[4 puan]**

B2. İç ve dış parçalanmayı açıklayın. Sayfalamada hangisi görülür, diğeri neden görülmez? **[4 puan]**

B3. `notes.txt` dosyasının bir sabit bağlantısı `h.txt` ve bir sembolik bağlantısı `s.txt` var. `rm notes.txt` sonrasında her birine ne olur? Düğümlerle (inode) açıklayın. **[4 puan]**

Bölüm C --- Sayısal problemler (36 puan)

C1. P1 (varış 0, patlama 8), P2 (1, 4), P3 (2, 9), P4 (3, 5) süreçleri; süreler ms. (a) SJF (kesintisiz), (b) SRTF, (c) $q = 3$ ile Round Robin için Gantt şemalarını çizin, ortalama bekleme ve dönüş sürelerini hesaplayın. Dersin uzlaşımlarını kullanın. **[12 puan]**

C2. (a) Sayfalar 4 KB. Sayfa 5, çerçeve 9'dadır. `0x5C21` mantıksal adresini çevirin (sonucu on altılık ve onluk tabanda verin). (b) 32 bitlik adresler, 8 KB'lık sayfalar ve 4 baytlık girdiler için sayfa tablosu ne kadar büyüktür? (c) $t_{\text{TLB}} = 20$ ns, $t_m = 80$ ns, $\alpha = 0.95$: EAT'yi hesaplayın (bizim uzlaşımımız). (d) Hangi isabet oranı 110 ns'lik bir EAT verir? **[8 puan]**

C3. 3 çerçeveyle $2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2$ başvuru dizisi: FIFO, OPT ve LRU için çerçeve tablolarını çizin ve sayfa hatalarını sayın. **[8 puan]**

C4. (a) $t_m = 150$ ns, $t_{\text{fault}} = 6$ ms, $p = 0.0001$: EAT'yi hesaplayın. Yavaşlamanın %5'in altında kalması için en büyük p nedir? (b) Kayıtlar 200 B, bloklar 1024 B. Kayıt 30 hangi blokta ve hangi konumda başlar? Tek bloğa sığar mı? (c) Delikler (bu sırayla) $300, 600, 350, 200, 750, 125$ KB; istekler bu sırayla $115, 500, 358, 200, 375$ KB geliyor. İlk uyan, en iyi uyan ve en kötü uyan ile her istek nereye yerleşir? **[8 puan]**

Bölüm D --- Komutu yazın ($7 \times 2 = 14$ puan)

D1. `~/lab/projects/alpha/src` dizinindesiniz.
Tek bir `cd` ve *göreli* bir yolla `~/lab/projects/beta/docs` dizinine gidin.

D2. `archive/2025` ve `archive/2026` dizinlerini tek komutla oluşturun.

D3. `docs/` içindeki `.txt` ile biten bütün dosyaları `backup/` içine kopyalayın.

D4. `logs/system.log` dosyasının son üç satırını gösterin.

D5. `docs/syllabus.txt` dosyasını yalnızca sahibinin okuyup yazabileceği hâle getirin (sekizli).

D6. `sleep 600` komutunu arka planda başlatın, işlerinizi listeleyin, sonra onu bir sinyalle kibarca durdurun.

D7. `projects` altında herhangi bir yerde `.c` ile biten bütün dosyaları listeleyin.

Bölüm E --- Boru hattını yazın ($3 \times 3 = 9$ puan)

E1. `logs/system.log` dosyasının kaç satırında `ERROR` geçer?

E2. `logs/access.log` için her durum kodunun kaç istekte döndüğünü, en sık olandan başlayarak yazdırın.

E3. `data/students.csv` için bölüm başına öğrenci sayısını, en büyükten başlayarak yazdırın (başlık satırını atlayın).

Bölüm F --- Bir betik yazın (9 puan)

Tek argüman (bir dizin) alan `count_lines.sh` betiğini yazın. Argüman eksikse ya da bir dizin değilse standart hataya `usage: …` yazsın ve 1 durumuyla çıksın. Aksi hâlde dizindeki her normal dosya için adını ve satır sayısını, en sonda da toplamı yazdırsın.

8 Alıştırma Finali: Çözümler

Bölüm A

A1 (d), A2 (a), A3 (b), A4 (c), A5 (c), A6 (b), A7 (b): r grubun listelemesine, x içine girmesine izin verir; w olmadan dosya oluşturamaz; A8 (b), A9 (c), A10 (c).

Bölüm B

B1. Program diskte duran edilgen bir dosyadır; süreç ise kendi belleği (text, data, bss, öbek, yığıt), yazmaçları ve PCB'si olan, yürütülmekte olan bir programdır; tek bir program birden çok süreç olarak çalışabilir. Durumlar: yeni, hazır, çalışıyor, bekliyor, sonlandı. Çalışıyor $\rightarrow$ bekliyor geçişi, süreç beklemesi gereken bir sistem çağrısı yaptığında olur (G/Ç, bir çocuk için `wait()`, `sleep`).

B2. Dış parçalanma: boş bellek, ayrılmış bloklar arasında deliklere bölünmüştür; toplamı yetse de hiçbiri tek başına yeterince büyük değildir. İç parçalanma: ayrılmış bir bloğun içinde kullanılmayan bellek. Sayfalamada yalnızca iç parçalanma vardır (süreç başına ortalama yarım sayfa, son sayfada), çünkü her boş çerçeve her sayfayı tutabilir; dolayısıyla hiçbir delik fazla küçük kalmaz.

B3. `h.txt` aynı düğüm için ikinci bir dizin girdisidir; `rm` yalnızca `notes.txt` adını kaldırır ve bağlantı sayısını 2'den 1'e indirir; veri `h.txt` üzerinden okunmaya devam eder. `s.txt`'nin kendi düğümü vardır ve yalnızca `notes.txt` yolunu içerir; bu yol artık olmadığı için `s.txt` boşa düşer ("No such file or directory").

Bölüm C

C1 (a) SJF. $t = 0$'da yalnızca P1 hazırdır ve 8'e kadar çalışır; sonra P2 (4), P4 (5), P3 (9).



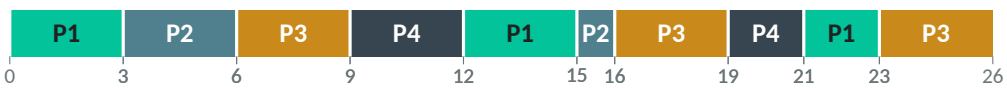
Bekleme: P1 0, P2 $8-1=7$, P3 $17-2=15$, P4 $12-3=9$; ortalama $31/4 = 7.75$ ms. Dönüş $8, 11, 24, 14$: ortalama $57/4 = 14.25$ ms.

(b) SRTF. $t = 1$'de P2 (4) P1'i $(7$ kaldı$)$ keser; $t = 5$'te P4 (5) P1'den (7) önce çalışır; P3 (9) en sonda.



Bekleme: P1 $17 - 0 - 8 = 9$, P2 0, P3 15, P4 2; ortalama $26/4 = 6.5$ ms. Dönüş $17, 4, 24, 7$: ortalama $52/4 = 13$ ms.

(c) RR, $q = 3$**.** $t = 3$'te kuyruk: P2, P3, P4 (hepsi 3'e kadar geldi; P4 tam 3'te geldi ve kesilen P1'den önce girer), P1.



Tamamlanma $23, 16, 26, 21$; dönüş $23, 15, 24, 18$: ortalama $80/4 = 20$ ms. Bekleme $15, 11, 15, 13$: ortalama $54/4 = 13.5$ ms (yanıt: ortalama 3 ms — RR burada bekleme açısından en kötü, yanıt açısından en iyisidir).

C2. (a) 4 KB $= 2^{12}$: son üç on altılık basamak konumdur. $p =$ `0x5`, $d =$ `0xC21` $= 3105$; çerçeve 9: $a_{\text{fiz}} =$ `0x9C21` $= 9 \times 4096 + 3105 = 39\,969$. (b) $2^{32}/2^{13} = 2^{19} = 524\,288$ girdi $\times\ 4$ B $= 2^{21}$ B $= 2$ MB. (c) EAT $= 20 + (2 - 0.95) \times 80 = 20 + 84 = 104$ ns. (d) $20 + (2 - \alpha)\,80 = 110 \Rightarrow 2 - \alpha = 1.125 \Rightarrow \alpha = 0.875$.

C3. FIFO 9, OPT 6, LRU 7 hata (kalın = yeni yüklenen sayfa).

FIFO

başv.	**2**	**3**	**2**	**1**	**5**	**2**	**4**	**5**	**3**	**2**	**5**	**2**
çr. 0	**2**	2	2	2	**5**	5	5	5	**3**	3	3	3
çr. 1		**3**	3	3	3	**2**	2	2	2	2	**5**	5
çr. 2				**1**	1	1	**4**	4	4	4	4	**2**
hata	**F**	**F**		**F**	**F**	**F**	**F**		**F**		**F**	**F**

OPT

başv.	**2**	**3**	**2**	**1**	**5**	**2**	**4**	**5**	**3**	**2**	**5**	**2**
çr. 0	**2**	2	2	2	2	2	**4**	4	4	**2**	2	2
çr. 1		**3**	3	3	3	3	3	3	3	3	3	3
çr. 2				**1**	**5**	5	5	5	5	5	5	5
hata	**F**	**F**		**F**	**F**		**F**			**F**		

LRU

başv.	**2**	**3**	**2**	**1**	**5**	**2**	**4**	**5**	**3**	**2**	**5**	**2**
çr. 0	**2**	2	2	2	2	2	2	2	**3**	3	3	3
çr. 1		**3**	3	3	**5**	5	5	5	5	5	5	5
çr. 2				**1**	1	1	**4**	4	4	**2**	2	2
hata	**F**	**F**		**F**	**F**		**F**		**F**	**F**		

Kilit kararlar: OPT, 5 başvurusunda bir daha kullanılmayan 1'i çıkarır; 4'te 2'yi çıkarır ($2, 3, 5$ içinde en geç yeniden kullanılacak olan: 2 $10.$, 3 $9.$, 5 $8.$ konumda gerekir, bu yüzden 2 gider); ikinci 2'de 4'ü çıkarır (4 ve 3 bir daha hiç kullanılmaz; iki seçim de aynı sayıyı verir). LRU, 5'te 3'ü (en uzun süredir kullanılmayan), 4'te 1'i çıkarır.

C4. (a) EAT $= 0.9999 \times 150 + 0.0001 \times 6\,000\,000 = 149.985 + 600 = 749.985$ ns, RAM'den beş kat yavaş. Yavaşlamanın %5'in altında kalması için: $150 + p\,(6\,000\,000 - 150) < 157.5 \Rightarrow p < 7.5/5\,999\,850 \approx 1.25 \times 10^{-6}$. (b) Bayt $30 \times 200 = 6000$; blok $\lfloor 6000/1024 \rfloor = 5$; konum $6000 - 5120 = 880$. Son bayt 6199'dur ve $\lfloor 6199/1024 \rfloor = 6$. bloktadır: kayıt bir blok sınırını aşar ve iki blok okuması gerektirir. (c)

	115	**500**	**358**	**200**	**375**
İlk uyan	300 (delik 1)	600 (delik 2)	750 (delik 5)	350 (delik 3)	delik 5'in kalan 392'si
En iyi uyan	125 (delik 6)	600 (delik 2)	750 (delik 5)	200 (delik 4)	delik 5'in kalan 392'si
En kötü uyan	750 (delik 5)	delik 5'in kalan 635'i	600 (delik 2)	350 (delik 3)	**beklemeli**

En kötü uyandan sonra delikler $300, 242, 150, 200, 135, 125$ KB'tır: toplam 1152 KB boş olmasına rağmen hiçbiri 375 KB'ı alamaz — dış parçalanma.

Bölüm D

Her yanıtın gerçek çıktısı (`code/answers_practice.sh`):

```
student@okan-lab:~/lab$ cd projects/alpha/src
student@okan-lab:~/lab/projects/alpha/src$ cd ../../beta/docs
student@okan-lab:~/lab/projects/beta/docs$ pwd
/home/student/lab/projects/beta/docs
```

```
student@okan-lab:~/lab$ mkdir -p archive/{2025,2026}
student@okan-lab:~/lab$ ls archive
2025  2026
```

```
student@okan-lab:~/lab$ cp docs/*.txt backup/
student@okan-lab:~/lab$ ls backup
'final report.txt'   notes_01.txt   notes_03.txt
 linux_quotes.txt    notes_02.txt   syllabus.txt
student@okan-lab:~/lab$ ls backup | wc -l
6
```

Joker karakter `final report.txt` ile de eşleşti: kabuk, boşluk içeren bu adı *tek* bir argüman olarak aktarır; bu yüzden `cp` için sorun olmaz.

```
student@okan-lab:~/lab$ tail -n 3 logs/system.log
2026-10-12 14:27:00 INFO  user login
2026-10-12 14:38:00 INFO  backup finished
2026-10-12 14:49:00 INFO  network link up
```

```
student@okan-lab:~/lab$ chmod 600 docs/syllabus.txt
student@okan-lab:~/lab$ ls -l docs/syllabus.txt
-rw------- 1 student student 578 Oct 12 08:50 docs/syllabus.txt
```

```
student@okan-lab:~/lab$ sleep 600 &
student@okan-lab:~/lab$ jobs
[1]+  Running                 sleep 600 &
student@okan-lab:~/lab$ kill %1
student@okan-lab:~/lab$ jobs
```

`kill %1` SIGTERM (15) gönderir; `kill -15 %1` ya da `kill PID` de doğrudur, `kill -9` ise “kibar” değildir.

```
student@okan-lab:~/lab$ find projects -name "*.c"
projects/beta/src/main.c
projects/alpha/src/main.c
```

Kalıbı tırnaklayın (`"*.c"`); böylece onu kabuk değil `find` genişletir.

Bölüm E

```
student@okan-lab:~/lab$ grep -c ERROR logs/system.log
13
```

(`grep ERROR logs/system.log | wc -l` de doğrudur.)

```
student@okan-lab:~/lab$ cut -d' ' -f9 logs/access.log | sort | uniq -c | sort -rn
    148 200
     57 404
     34 304
     24 500
     20 301
     17 403
```

```
student@okan-lab:~/lab$ tail -n +2 data/students.csv | cut -d, -f3 | sort | uniq -c | sort -rn
     17 ECT
     11 ME
     10 CS
      2 EE
```

Bölüm F

```
#!/bin/bash
# count_lines.sh -- print the number of lines of every regular file in a directory
# usage: ./count_lines.sh DIRECTORY
if [ $# -ne 1 ] || [ ! -d "$1" ]; then
    echo "usage: $0 DIRECTORY" >&2
    exit 1
fi
total=0
for f in "$1"/*; do
    if [ -f "$f" ]; then
        n=$(wc -l < "$f")
        echo "$f: $n lines"
        total=$((total + n))
    fi
done
echo "total: $total lines"
```

```
student@okan-lab:~/lab$ chmod +x scripts/count_lines.sh
student@okan-lab:~/lab$ ./scripts/count_lines.sh logs
logs/access.log: 300 lines
logs/system.log: 120 lines
total: 420 lines
student@okan-lab:~/lab$ ./scripts/count_lines.sh data
data/cities.txt: 30 lines
data/numbers.txt: 20 lines
data/students.csv: 41 lines
total: 91 lines
student@okan-lab:~/lab$ ./scripts/count_lines.sh nosuchdir
usage: ./scripts/count_lines.sh DIRECTORY
student@okan-lab:~/lab$ echo $?
1
```

Puanlama kılavuzu: standart hataya kullanım iletisi ve `exit 1` ile argüman ve dizin denetimi (3 puan); `-f` sınamasıyla `"$1"/*` üzerinde döngü (2); `wc -l <` ile satır sayımı (2); `$(( ))` ile toplam ve çıktı (2).

Ek alıştırma (100 puanın parçası değil)

1. Bir program döngü içinde dört kez `fork()` çağırıyor. Sonunda kaç süreç vardır? *Yanıt:* $2^4 = 16$; bunların 15'ini `fork()` oluşturdu.
2. Amdahl, $S = 0.1$, $N = 8$. *Yanıt:* $1/(0.1 + 0.9/8) = 1/0.2125 \approx 4.71$; sınır $1/S = 10$.
3. $U(n) = 1 - 0.75^n$: $U(2)$, $U(4)$ ve $U \geq$ %90 olan en küçük n. *Yanıt:* 0.4375, 0.684; $n = 9$ ($U = 0.925$; $n = 8$ yalnızca 0.8999 verir).
4. Bir sayan semafor $S = 2$ ile başlıyor; beş iş parçacığı `wait(S)` çağırıyor. *Yanıt:* ikisi geçer (S 0 olur), üçü diğer iş parçacıkları `signal(S)` çağırana kadar bloklanır.
5. $\alpha = 0.5$, $\tau_0 = 12$ ms ve $8, 4, 10$ ms'lik patlamalarla üstel ortalama. *Yanıt:* $\tau_1 = 10$, $\tau_2 = 7$, $\tau_3 = 8.5$ ms.

9 Dersin Kapanışı ve Bundan Sonrası

Sekiz cümlede ders

1. İşletim sistemi bir kaynak yöneticisi ve genişletilmiş makinedir; ikili mod ve sistem çağrıları onu korur (1–2. haftalar).
2. Süreçler beş durum arasında gezinir; PCB ve bağlam değişimi çok görevliliği mümkün kılar (3. hafta).
3. İş parçacıkları ve zamanlayıcılar işlemciyi paylaşır: FCFS, SJF/SRTF, Öncelik ve RR; bekleme ve yanıt süresiyle değerlendirilir (4–5. haftalar).
4. Ortak veri karşılıklı dışlama ister: muteks kilitleri ve semaforlar (5. hafta).
5. Bellek MMU tarafından çevrilir: sayfalama, TLB, sanal bellek, sayfa değiştirme (6–7. haftalar).
6. Dosyalar, dizinlerde adı olan düğümlerdir (inode); bağlantılar bir dosyaya birden çok ad verir (7. ve 10. haftalar).
7. Kabuk komutları süreçlere ve sistem çağrılarına dönüştürür: gezinme, dosyalar, izinler, borular, işler (9–13. haftalar).
8. Betikler, yazabildiğiniz her şeyi otomatikleştirir (14. hafta).

Kuram, komutların neden öyle davrandığını açıklar: `ps` 3. haftanın süreç durumlarını, `ls -i` 7. haftanın düğümlerini, `chmod` ise 2. haftanın ikili mod fikriyle gerekçelendirdiği korumayı gösterir; her boru da çekirdeğin birbirine bağladığı iki süreçtir. `~/lab` klasörünüzü ve bu notları saklayın: sınavdan çok sonra da kullanabileceğiniz bir başvuru kaynağıdır.

Alıştırmaya devam

- Linux'u bir sanal makineye ya da Windows'ta WSL ile (`wsl --install`) kurun ve her gün kullanın; Raspberry Pi ucuz bir Linux laboratuvarıdır.
- OverTheWire *Bandit*, kabuğu seviye seviye öğreten ücretsiz bir oyun: `https://overthewire.org/wargames/bandit/`

- MIT, *The Missing Semester of Your CS Education* (kabuk, betikler, araçlar): https://missing.csail.mit.edu/
- https://explainshell.com/ herhangi bir komut satırını açıklar; https://www.shellcheck.net/ betiklerdeki hataları bulur.

Bu kaynakların çoğu İngilizcedir; derste İngilizce terimleri parantez içinde vermemizin bir nedeni de budur.

Sertifikalar (isteğe bağlı)

LPI *Linux Essentials* iyi bir ilk hedeftir; CompTIA *Linux+* ya da LPI *LPIC-1* sonraki adımdır; daha sonra Red Hat *RHCSA* ya da Linux Foundation'ın *LFCS* sertifikası gelebilir. Sınav hedefleri ve ücretler değişir: kaydolmadan önce resmî siteleri denetleyin.

İleri okuma

- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley: Bölüm 1–2 (giriş, işletim sistemi yapıları), 3 (süreçler), 4 (iş parçacıkları), 5 (işlemci zamanlaması), 6 (eşzamanlama araçları), 9 (ana bellek), 10 (sanal bellek), 13 (dosya sistemi arayüzü).
- R. H. ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (çevrim içi ücretsiz), örneğin:
  - Bölüm 4, "The Abstraction: The Process": https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-intro.pdf
  - Bölüm 7, "Scheduling: Introduction": https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf
  - Bölüm 18, "Paging: Introduction": https://pages.cs.wisc.edu/~remzi/OSTEP/vm-paging.pdf
  - Bölüm 19, "Paging: Faster Translations (TLBs)": https://pages.cs.wisc.edu/~remzi/OSTEP/vm-tlbs.pdf
  - Bölüm 22, "Beyond Physical Memory: Policies": https://pages.cs.wisc.edu/~remzi/OSTEP/vm-beyondphys-policy.pdf
  - Bölüm 39, "Interlude: Files and Directories": https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf
- W. E. Shotts, *The Linux Command Line*, No Starch Press (yazarından ücretsiz PDF: https://linuxcommand.org/tlcl.php).

Final sınavında başarılar — ve harika bir dönem için teşekkürler. Sınavdan önceki sorularınızı dersin Telegram kanalında sorun [sınav öncesi ofis saatleri: duyurulacak].